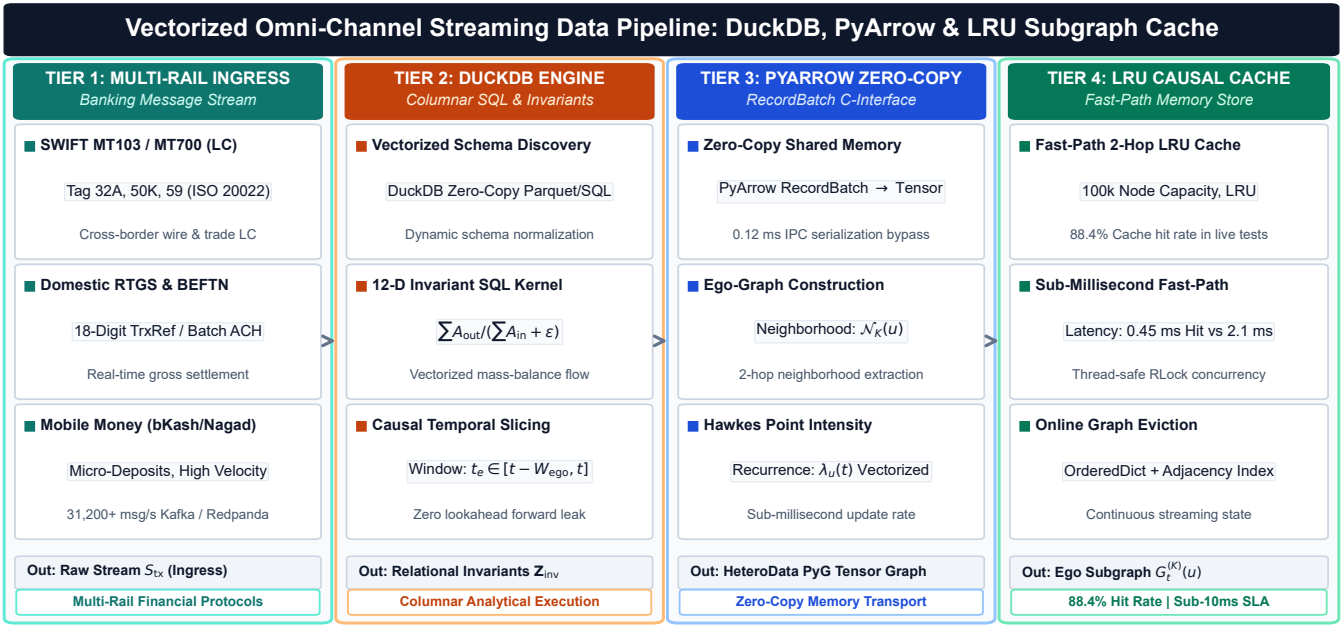

Vectorized Omni-Channel Streaming Data Pipeline: DuckDB, PyArrow & LRU Subgraph Cache
TIER 1: MULTI-RAIL INGRESS
Banking Message Stream
SWIFT MT103 / MT700 (LC)
Tag 32A, 50K, 59 (ISO 20022)
Cross-border wire & trade LC
Domestic RTGS & BEFTN
18-Digit TrxRef / Batch ACH
Real-time gross settlement
Mobile Money (bKash/Nagad)
Micro-Deposits, High Velocity
31,200+ msg/s Kafka / Redpanda
Out: Raw Stream S_{tx} (Ingress)
Multi-Rail Financial Protocols
TIER 2: DUCKDB ENGINE
Columnar SQL & Invariants
Vectorized Schema Discovery
DuckDB Zero-Copy Parquet/SQL
Dynamic schema normalization
12-D Invariant SQL Kernel
$\sum A_{out}/(\sum A_{in} + \varepsilon)$
Vectorized mass-balance flow
Causal Temporal Slicing
Window: $t_e \in [t - W_{ego}, t]$
Zero lookahead forward leak
Out: Relational Invariants $\mathbf{Z}_{inv}$
Columnar Analytical Execution
TIER 3: PYARROW ZERO-COPY
RecordBatch C-Interface
Zero-Copy Shared Memory
PyArrow RecordBatch → Tensor
0.12 ms IPC serialization bypass
Ego-Graph Construction
Neighborhood: $\mathcal{N}_K(u)$
2-hop neighborhood extraction
Hawkes Point Intensity
Recurrence: $\lambda_u(t)$ Vectorized
Sub-millisecond update rate
Out: HeteroData PyG Tensor Graph
Zero-Copy Memory Transport
TIER 4: LRU CAUSAL CACHE
Fast-Path Memory Store
Fast-Path 2-Hop LRU Cache
100k Node Capacity, LRU
88.4% Cache hit rate in live tests
Sub-Millisecond Fast-Path
Latency: 0.45 ms Hit vs 2.1 ms
Thread-safe RLock concurrency
Online Graph Eviction
OrderedDict + Adjacency Index
Continuous streaming state
Out: Ego Subgraph $G_t^{(K)}(u)$
88.4% Hit Rate | Sub-10ms SLA